

LABORATÓRIO TPSE II



Lab 01: Construindo Toolchain

Prof. Francisco Helder

15 de outubro de 2024

Nesta atividade iremos configurar e compilar nosso próprio toolchain usando a ferramenta crosstool-ng. Este será o toolchain que depois utilizaremos para compilar o bootloader, o kernel, as bibliotecas e aplicações para a placa BeagleBone Black, que desenvolveremos no decorrer do curso.

1 Aprendendo a trabalhar com Crosstool-ng

Pacotes necessários para realização deste laboratório:

```
$ sudo apt install build-essential git autoconf bison flex texinfo help2man gawk libtool-bin libncurses5-dev unzip
```

1.1 Obtendo o Crosstool-ng

Vamos baixar as fontes do Crosstool-ng, através do seu repositório git, e mudar para uma versão nova e estável:

```
$ git clone https://github.com/crosstool-ng/crosstool-ng
$ cd crosstool-ng/
$ git checkout crosstool-ng-1.26.0
```

1.2 Construindo e Instalando o Crosstool-ng

Como não estamos construindo o Crosstool-ng a partir de um arquivo de lançamento, mas de um repositório git, primeiro precisamos gerar um script de configuração e, de forma mais geral, todos os arquivos gerados que são enviados no arquivo de origem para um release:

```
$ ./bootstrap
```

Adicione à variável de ambiente PATH o caminho dos binários do toolchain. Desta forma, teremos acesso às ferramentas de compilação do toolchain no terminal corrente.

```
$ export PATH=/home/XXX/labs/toolchain/gcc-linaro-arm/bin:$PATH
```

Podemos então instalar o Crosstool-ng globalmente no sistema ou mantê-lo localmente em seu diretório de download. Escolheremos a última solução. Conforme documentado em <https://crosstool-ng.github.io/docs/install/#hackers-way>, então faça como seguinte:

```
$ ./configure --enable-local
$ make
```

Então você pode obter ajuda do Crosstool-ng executando:

```
$ ./ct-ng help
```

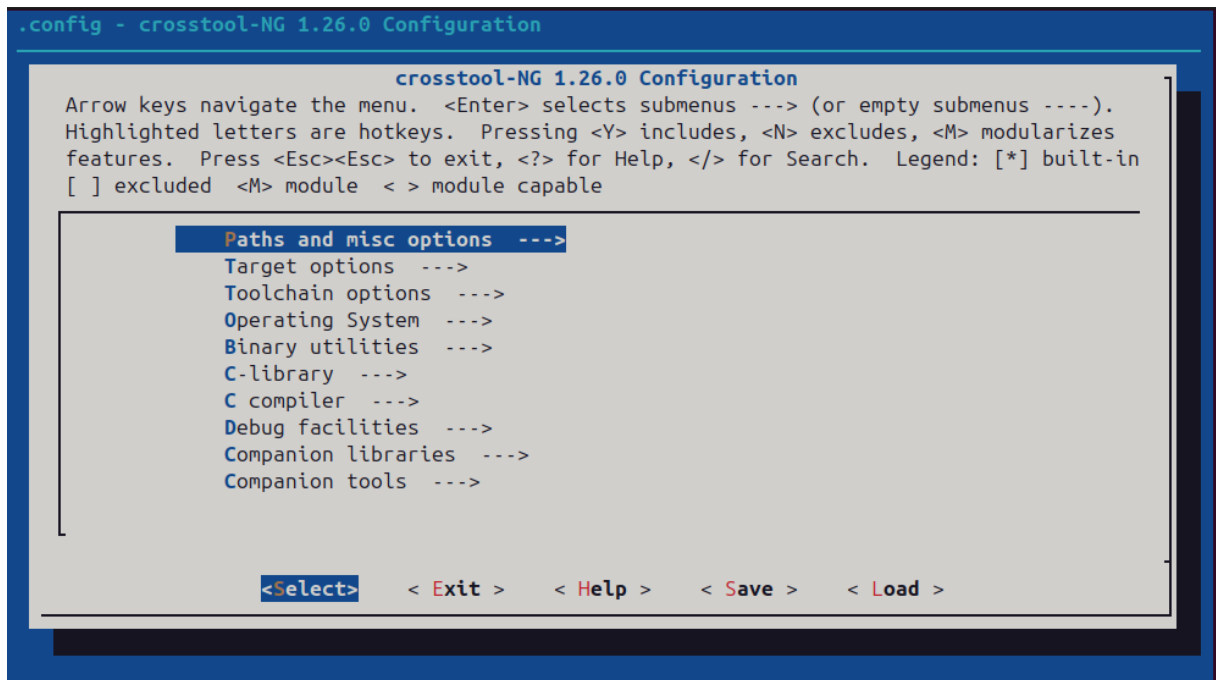
1.3 Configure o Toolchain

Uma única instalação do Crosstool-ng permite produzir quantas Toolchains que você quiser, para diferentes arquiteturas, com diferentes bibliotecas C e diferentes versões dos vários componentes.

O Crosstool-ng vem com um conjunto de arquivos de configuração prontos para várias configurações típicas: o Crosstool-ng os chama de **samples**. Eles podem ser listados usando `./ct-ng list-samples`.

Vamos carregar o sample do Cortex A8. Carregue-o com o comando `./ct-ng`. Então, para refinar a configuração, vamos executar a funcionalidade **menuconfig**:

```
$ ./ct-ng menuconfig
```



Em **Path and misc options**:

- Se ainda não estiver definido, habilite a opção **Try features marked as EXPERIMENTAL**.

Em **Target options**:

- Configure a opção **Use specific FPU (ARCH_FPU)** para **vfpv3**.
- Configure a opção **Floating point** para **hardware (FPU)**.

Em **Toolchain options**:

- Configure a opção **Tuple's vendor string** para **tpse2**.
- Configure a opção **Tuple's alias** para **arm-linux**. Dessa forma, poderemos usar o compilador como **arm-linux-gcc** em vez de **arm-training-linux-musleabihf-gcc**, que é muito mais longo para digitar.

Em **Operating System**:

- Defina a versão do Linux para a mais próxima, para **6.4**. É importante que os headers do kernel usados no Toolchain não sejam mais recentes do que o kernel que será executado na placa (v6.4).

Em **C-library**:

- Caso já não esteja configurado, configure a opção **C library** para **musl**.
- Mantenha a versão padrão proposta.

Em **C compiler**:

- Configure a opção **Version of gcc** para 13.2.0.
- Certifique-se de que C++ esteja habilitado.

Em **Debug facilities**:

- Remova todas as opções aqui. Algumas ferramentas de depuração podem ser fornecidas no Toolchain, mas também podem ser construídas por ferramentas de construção de sistema de arquivos.

1.4 Produzindo o Toolchain

Agora é simplesmente buildar com o comando `ct-ng`:

```
$ ./ct-ng build
```

O Toolchain será instalada por padrão em `$HOME/x-tools/`. Mas para nossas práticas, você deve alterar o caminho onde o Crosstool-ng deve instalar o Toolchain (Exemplo: colocar em um diretório de trabalho da disciplina TPSEII como `$XXX/toolchain`).

1.5 Testando o toolchain

Agora você pode testar seu Toolchain adicionando `$HOME/x-tools/arm-tpse2-linux-musleabihf/bin/` à sua variável de ambiente `PATH` e compilando o programa simples `main.c` no diretório principal do seu laboratório com `arm-linux-gcc`:

```
1 #include <stdio.h>
2 int main() {
3     printf("Hello_embedded_world!\n");
4     return(0);
5 }
```

Compile a aplicação para ARM com o toolchain gerado pelo `crosstool-ng`:

```
$ arm-linux-gcc main.c -o main
```

Use o comando `file` para verificar as informações do arquivo. Perceba que a aplicação foi compilada para a arquitetura ARM:

```
$ file main
```

main: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 2.6.32, not stripped

1.5.1 Linkando a Aplicação Estaticamente

Compile novamente a aplicação para ARM, porém linkando com as bibliotecas do sistema de forma estática. Depois compare o tamanho do binário gerado com a mesma aplicação compilada e linkada dinamicamente.

Dica: para identificar os parâmetros necessários para compilar uma aplicação estaticamente, dê uma olhada na página de manual do gcc:

```
$ man gcc
```

Para fazer uma busca dentro de uma página de manual, pressione “/” e digite a(s) palavra(s) que deseja procurar. Pressionando a letra ‘n’ (next) você pode avançar para a próxima palavra na busca.

2 Atividades Práticas

pratica 1:

Cross-compile uma aplicação “Hello World!” com a lib *stdio* linkado dinâmica e estaticamente, envie e execute no Linux que está rodando na BleagleBone Black.